

1. Revision of Basics of Python

1.1 What is Python?

Python is a high-level, general-purpose, interpreted programming language. It was created by Guido van Rossum and first released in 1991. Python is designed with simplicity and readability in mind, using English-like syntax that makes it easy to learn and use.

Features of Python

- Simple and Easy to Learn – Python has a clean, simple syntax close to English.
- Interpreted – Python runs line by line, making debugging easier.
- Platform Independent – Python programs can run on Windows, Mac, Linux without changes.
- Open Source – Python is free to download and use.
- High-level Language – Python handles memory management automatically.
- Dynamically Typed – No need to declare the data type of variables.
- Object-Oriented – Python supports OOP concepts like classes and objects.
- Large Standard Library – Python comes with many built-in modules and libraries.

Advantages of Python

- Fast to write and easy to read.
- Large community and support.
- Used in many fields: web, AI, data science, automation.
- Fewer lines of code compared to other languages.

Applications of Python

Application Area	Examples
Web Development	Django, Flask frameworks for websites
Software Development	Building desktop applications
Data Science	Analyzing and processing large datasets
Artificial Intelligence	Machine learning, neural networks, deep learning
Automation	Writing scripts to automate repetitive tasks
Game Development	Simple 2D/3D games with Pygame library

1.2 Fundamental Syntax of Python

Python Keywords

Keywords are reserved words in Python that have a special meaning. They cannot be used as variable names.

Examples: if, else, elif, for, while, def, return, import, True, False, None, and, or, not, in, is, class, try, except, finally, break, continue, pass, lambda, with

 **Exam Tip:** Python is case-sensitive. 'if' is a keyword but 'IF' is not.

Indentation

Python uses indentation (spaces or tabs at the beginning of a line) to define blocks of code. Unlike other languages that use curly braces {}, Python uses indentation to group statements.

```
if x > 5:
    print('x is greater than 5') # indented block
    print('This also belongs to if')
print('This is outside the if block') # not indented
```

 **Exam Tip:** Always use 4 spaces (or 1 tab) for indentation. Inconsistent indentation causes IndentationError.

Comments

Comments are lines that Python ignores during execution. They are used to explain code.

Single-line Comment — using

```
# This is a single-line comment
x = 5 # This sets x to 5
```

Multi-line Comment — using triple quotes

```
"""
This is a
multi-line comment
"""
```

Case Sensitivity

Python is case-sensitive. 'name', 'Name', and 'NAME' are three different variables.

Statements

A statement is a single instruction that Python can execute. Examples: `x = 5`, `print('Hello')`, `if x > 0:`

Code Blocks

A code block is a group of statements that belong together (e.g., inside a function or loop). In Python, code blocks are defined by indentation.

2. Input and Output

2.1 input() — Taking Input from User

The input() function is used to take input from the user through the keyboard. It always returns a string.

Syntax

```
variable = input('message to user')
```

Examples

```
name = input('Enter your name: ')
print('Hello', name)
```

```
# Output (if user types 'Aarav'):
# Enter your name: Aarav
# Hello Aarav
```

 **Exam Tip:** input() always returns a string. To use the input as a number, you must convert it.

2.2 print() — Displaying Output

The print() function is used to display output on the screen.

Syntax

```
print(value1, value2, ...)
```

Examples

```
print('Hello, World!')
print('Sum =', 10 + 5)
print('Name:', 'Alice', 'Age:', 20)
```

2.3 Type Conversion with Input

Since input() always returns a string, we must convert it to the required type.

Function	Purpose and Example
int(input())	Converts input to integer: age = int(input('Enter age: '))
float(input())	Converts input to float: price = float(input('Enter price: '))
str(input())	Keeps as string (default): name = str(input('Enter name: '))

Example:

```
num1 = int(input('Enter first number: '))
num2 = int(input('Enter second number: '))
print('Sum =', num1 + num2)
```

3. Data Types and Variables

3.1 Variables

A variable is a named location in memory that stores a value. In Python, variables are created when you assign a value to them.

Rules for Naming Variables

- Must start with a letter (a-z, A-Z) or underscore (_).
- Cannot start with a number.
- Can contain letters, numbers, and underscores.
- Cannot use Python keywords as variable names.
- Case-sensitive: 'age' and 'Age' are different.

Examples:

```
name = 'Alice'    # valid
age = 16          # valid
_score = 95       # valid
2name = 'Bob'     # INVALID – starts with number
for = 10          # INVALID – 'for' is a keyword
```

3.2 Data Types

A data type defines the type of value a variable can hold. Python automatically detects the data type.

A. Numeric Data Types

```
x = 10           # int (integer)
y = 3.14         # float (decimal)
z = 2 + 3j       # complex (real + imaginary)
```

B. Text Data Type

```
name = 'Hello'   # str (string)
msg = "Python"   # also a string
```

C. Boolean Data Type

```
is_student = True # bool
is_adult = False  # bool
```

D. Sequence Data Types

```
my_list = [1, 2, 3, 'apple'] # list (mutable)
my_tuple = (10, 20, 30)      # tuple (immutable)
```

E. Set Data Type

```
my_set = {1, 2, 3, 4} # set (unordered, unique elements)
```

F. Mapping Data Type

```
my_dict = {'name': 'Alice', 'age': 16} # dict
```

G. None Type

```
x = None # represents 'no value' or 'empty'
```

Data Type	Category	Example	use type()
int	Numeric	x = 10	type(x) → int
float	Numeric	x = 3.14	type(x) → float
complex	Numeric	x = 2+3j	type(x) → complex
str	Text	x = 'Hi'	type(x) → str
bool	Boolean	x = True	type(x) → bool
list	Sequence	x = [1,2,3]	type(x) → list
tuple	Sequence	x = (1,2,3)	type(x) → tuple
set	Set	x = {1,2,3}	type(x) → set
dict	Mapping	x = {'a':1}	type(x) → dict
None	None Type	x = None	type(x) → NoneType

 **Exam Tip:** Use type(variable) to check the data type of any variable.

4. Operators and Expressions

4.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations.

Operator + Name	Example and Output
+ (Addition)	$5 + 3 \rightarrow 8$
- (Subtraction)	$10 - 4 \rightarrow 6$
* (Multiplication)	$4 * 3 \rightarrow 12$
/ (Division)	$10 / 3 \rightarrow 3.3333$
// (Floor Division)	$10 // 3 \rightarrow 3$ (integer part only)
% (Modulus)	$10 \% 3 \rightarrow 1$ (remainder)
** (Exponent)	$2 ** 4 \rightarrow 16$ (2 raised to power 4)

Example program:

```
a = 15
b = 4
print('Add:', a + b)    # 19
print('Sub:', a - b)    # 11
print('Mul:', a * b)    # 60
print('Div:', a / b)    # 3.75
print('Floor:', a // b) # 3
print('Mod:', a % b)    # 3
print('Power:', a ** b) # 50625
```

 **Exam Tip:** Floor division (//) always gives an integer result by rounding down. Modulus (%) gives the remainder after division.

4.2 Relational (Comparison) Operators

Relational operators compare two values and return a Boolean result (True or False).

Operator	Meaning and Example
==	Equal To: $5 == 5 \rightarrow \text{True}$
!=	Not Equal To: $5 != 3 \rightarrow \text{True}$
>	Greater Than: $7 > 3 \rightarrow \text{True}$
<	Less Than: $2 < 9 \rightarrow \text{True}$
>=	Greater Than or Equal: $5 >= 5 \rightarrow \text{True}$
<=	Less Than or Equal: $3 <= 6 \rightarrow \text{True}$

```
x = 10
```

```
print(x > 5) # True
print(x == 10) # True
print(x != 10) # False
```

4.3 Logical Operators

Logical operators are used to combine multiple conditions.

Operator	Meaning and Example
and	True only if BOTH conditions are True: (5>2) and (3<10) → True
or	True if AT LEAST ONE condition is True: (5>2) or (3>10) → True
not	Reverses the result: not(5>2) → False

```
age = 20
print(age >= 18 and age <= 60) # True
print(age < 18 or age > 60) # False
print(not(age == 20)) # False
```

4.4 Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Meaning (example: x = 10)
=	Assign: x = 10 → x is 10
+=	Add and assign: x += 5 → x becomes 15
-=	Subtract and assign: x -= 3 → x becomes 7
*=	Multiply and assign: x *= 2 → x becomes 20
/=	Divide and assign: x /= 2 → x becomes 5.0
//=	Floor divide and assign: x //= 3 → x becomes 3
%=	Modulus and assign: x %= 3 → x becomes 1
**=	Exponent and assign: x **= 2 → x becomes 100

5. String Formatting

5.1 Format Specifiers (% formatting)

Format specifiers are special codes used inside a string with % operator to format output.

Specifier	Meaning and Example
%d	Integer: 'Age: %d' % 16 → 'Age: 16'
%f	Float: 'Price: %f' % 3.5 → 'Price: 3.500000'
%.2f	Float with 2 decimal places: '%.2f' % 3.14159 → '3.14'
%.nf	Float with n decimal places
%s	String: 'Hello %s' % 'Alice' → 'Hello Alice'
%x	Hexadecimal (lowercase): '%x' % 255 → 'ff'
%X	Hexadecimal (uppercase): '%X' % 255 → 'FF'
%o	Octal: '%o' % 8 → '10'

Example:

```
name = 'Alice'
age = 16
marks = 89.567
print('Name: %s, Age: %d, Marks: %.2f' % (name, age, marks))
# Output: Name: Alice, Age: 16, Marks: 89.57
```

5.2 f-Strings (Formatted String Literals)

f-Strings are the modern, preferred way to format strings in Python. They are easy to read and fast.

Syntax

```
f'text {variable} text'
```

Basic f-string:

```
name = 'Aarav'
print(f'Hello, {name}!')
# Output: Hello, Aarav!
```

Expressions inside f-string:

```
a = 10
b = 5
print(f'Sum = {a + b}')
# Output: Sum = 15
```

Formatting numbers:

```
pi = 3.14159
```



```
print(f'Pi = {pi:.2f}')  
# Output: Pi = 3.14
```

f-string with Dictionary:

```
student = {'name': 'Ram', 'grade': 'A'}  
print(f"Name: {student['name']], Grade: {student['grade']}")  
# Output: Name: Ram, Grade: A
```

Feature	% Formatting vs f-String
Syntax	'Hello %s' % name vs f'Hello {name}'
Readability	Less readable vs More readable
Expressions	Not possible vs Allowed inside {}
Recommended	Older style vs Modern, preferred

6. Conditional Statements

Conditional statements allow a program to make decisions. The program executes different blocks of code based on whether a condition is True or False.

6.1 if Statement

The if statement executes a block of code only when the given condition is True.

Syntax

```
if condition:
    # block of code (executed if condition is True)
```

Example:

```
age = 18
if age >= 18:
    print('You are eligible to vote.')
# Output: You are eligible to vote.
```

6.2 if-else Statement

The if-else statement provides two paths: one when the condition is True and one when it is False.

Syntax

```
if condition:
    # executed if True
else:
    # executed if False
```

Example:

```
num = int(input('Enter a number: '))
if num % 2 == 0:
    print('Even number')
else:
    print('Odd number')
```

6.3 if-elif-else Statement

Used when there are more than two conditions to check. elif means 'else if'.

Syntax

```
if condition1:
    # block 1
elif condition2:
    # block 2
elif condition3:
    # block 3
else:
    # default block
```

Example — Grade Checker:

```
marks = int(input('Enter marks: '))
if marks >= 90:
    print('Grade: A')
elif marks >= 80:
    print('Grade: B')
elif marks >= 70:
    print('Grade: C')
elif marks >= 60:
    print('Grade: D')
else:
    print('Grade: F (Fail)')
```

6.4 Nested if

A nested if is an if statement inside another if statement. It is used to check conditions within conditions.

Example:

```
num = int(input('Enter a number: '))
if num >= 0:
    if num == 0:
        print('Zero')
    else:
        print('Positive number')
else:
    print('Negative number')
```

 **Exam Tip:** Be careful with indentation in nested if. Each level of nesting adds one more indentation.

7. Python Collections

7.1 String

A string is a sequence of characters enclosed in single quotes ('), double quotes (""), or triple quotes ("""). Strings are immutable (cannot be changed after creation).

```
name = 'Python'
print(name[0])    # P (indexing)
print(name[1:4])  # yth (slicing)
print(len(name))  # 6 (length)
```

7.2 List

A list is an ordered, mutable (changeable) collection of items enclosed in square brackets []. It can contain items of different data types.

Features

- Ordered — items maintain their position.
- Mutable — items can be added, removed, or changed.
- Allows duplicate elements.
- Can contain mixed data types.

```
fruits = ['apple', 'banana', 'mango']
fruits.append('orange')    # add item
fruits[0] = 'grape'        # change item
print(fruits)              # ['grape', 'banana', 'mango', 'orange']
```

7.3 Tuple

A tuple is an ordered, immutable (unchangeable) collection of items enclosed in parentheses (). Once created, a tuple cannot be modified.

Features

- Ordered — items maintain their position.
- Immutable — cannot add, remove, or change items.
- Allows duplicate elements.
- Faster than lists.

```
coordinates = (10, 20, 30)
print(coordinates[1]) # 20
# coordinates[1] = 50 ← This would cause a TypeError!
```

7.4 Dictionary

A dictionary is an unordered collection of key-value pairs enclosed in curly braces {}. Each key must be unique.

Features

- Stores data as key:value pairs.
- Keys must be unique and immutable (strings, numbers, tuples).
- Values can be of any data type.
- Mutable — can add, modify, or delete key-value pairs.

```
student = {'name': 'Priya', 'age': 16, 'grade': 'A'}
print(student['name'])    # Priya
student['age'] = 17        # update
student['school'] = 'ABC'  # add new key
print(student)
```

Feature	List	Tuple	Dictionary
Syntax	[1,2,3]	(1,2,3)	{k:v}
Mutability	Mutable	Immutable	Mutable
Order	Ordered	Ordered	Ordered (Python 3.7+)
Duplicates	Allowed	Allowed	Keys unique, values ok
Access	By index	By index	By key

8. Range Function

8.1 range()

The range() function generates a sequence of numbers. It is commonly used with for loops.

Syntax Variations

Syntax	Generates
range(stop)	Numbers from 0 to stop-1: range(5) → 0,1,2,3,4
range(start, stop)	Numbers from start to stop-1: range(2,6) → 2,3,4,5
range(start, stop, step)	With step: range(0,10,2) → 0,2,4,6,8

Examples:

```
for i in range(5):  
    print(i, end=' ')  
# Output: 0 1 2 3 4
```

```
for i in range(1, 6):  
    print(i, end=' ')  
# Output: 1 2 3 4 5
```

```
for i in range(0, 20, 5):  
    print(i, end=' ')  
# Output: 0 5 10 15
```

```
for i in range(10, 0, -2):  
    print(i, end=' ')  
# Output: 10 8 6 4 2 (negative step = countdown)
```

 **Exam Tip:** range(stop) is the same as range(0, stop, 1). The stop value is NEVER included in the sequence.

9. Loops

Loops are used to execute a block of code repeatedly. Python has two main types of loops: for and while.

9.1 for Loop

A for loop is used to iterate over a sequence (list, tuple, string, range, etc.) and execute a block of code for each item.

Syntax

```
for variable in sequence:  
    # block of code
```

Example 1 — Counting:

```
for i in range(1, 6):  
    print(i)  
# Output: 1 2 3 4 5 (each on new line)
```

Example 2 — Multiplication Table:

```
n = int(input('Enter a number: '))  
for i in range(1, 11):  
    print(f'{n} x {i} = {n * i}')
```

Example 3 — List Traversal:

```
fruits = ['apple', 'banana', 'mango']  
for fruit in fruits:  
    print(fruit)
```

9.2 while Loop

A while loop repeats a block of code as long as the given condition is True. It is used when the number of iterations is not known beforehand.

Syntax

```
while condition:  
    # block of code
```

Example 1 — Counting:

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1  
# Output: 1 2 3 4 5
```

Example 2 — Sum of numbers:

```
total = 0  
i = 1  
while i <= 100:  
    total += i  
    i += 1
```

```
print('Sum =', total) # Sum = 5050
```

Example 3 — User Controlled Loop:

```
ans = 'yes'
while ans == 'yes':
    name = input('Enter name: ')
    print('Hello', name)
    ans = input('Continue? (yes/no): ')
```

for Loop	while Loop
Used when number of iterations is known	Used when number of iterations is unknown
Iterates over a sequence	Runs until a condition becomes False
for i in range(5):	while i < 5:
More compact for sequences	More flexible for conditional loops

10. Loop Control Statements

10.1 break

The break statement immediately stops the loop and exits it, regardless of the condition.

```
for i in range(1, 11):
    if i == 5:
        break
    print(i)
# Output: 1 2 3 4 (stops before 5)
```

10.2 continue

The continue statement skips the rest of the current iteration and moves to the next iteration.

```
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
# Output: 1 3 5 7 9 (skips even numbers)
```

10.3 else with Loop

Python allows an else clause with loops. The else block executes only when the loop completes normally (not exited by break).

```
for i in range(1, 6):
    print(i)
else:
    print('Loop finished normally!')
# Output: 1 2 3 4 5
# Loop finished normally!
```

With break (else does NOT run):

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
else:
    print('This will NOT print because break was used')
# Output: 1 2
```

 **Exam Tip:** break, continue, and else can be used with both for and while loops.

11. Nested Loops

11.1 Definition

A nested loop is a loop inside another loop. The inner loop executes completely for every single iteration of the outer loop.

11.2 Examples

Example 1 — Number Pattern:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(j, end=' ')
    print()
# Output:
# 1 2 3
# 1 2 3
# 1 2 3
```

Example 2 — Star Pattern:

```
for i in range(1, 6):
    for j in range(i):
        print('*', end='')
    print()
# Output:
# *
# **
# ***
# ****
# *****
```

Example 3 — Multiplication Table (All):

```
for i in range(1, 6):
    for j in range(1, 6):
        print(f'{i}x{j}={i*j}', end='\t')
    print()
```

12. Built-in Library Functions

12.1 String Functions

Function	Description and Example
upper()	Converts to uppercase: 'hello'.upper() → 'HELLO'
lower()	Converts to lowercase: 'HELLO'.lower() → 'hello'
center(width)	Centers string: 'hi'.center(10) → ' hi '
len()	Returns length: len('Python') → 6

Examples:

```
s = 'Python Programming'
print(s.upper())      # PYTHON PROGRAMMING
print(s.lower())      # python programming
print(s.center(30, '-')) # -----Python Programming-----
print(len(s))         # 18
```

12.2 Mathematical Functions

Function	Description and Example
sum(list)	Sum of all elements: sum([1,2,3,4]) → 10
pow(x,y)	x to the power y: pow(2,10) → 1024
round(x, n)	Round to n decimals: round(3.14159, 2) → 3.14
abs(x)	Absolute value: abs(-15) → 15
sqrt(x)	Square root (from math): sqrt(144) → 12.0
int(x)	Convert to integer: int(9.9) → 9

```
import math
print(sum([10, 20, 30])) # 60
print(pow(3, 4))         # 81
print(round(7.8765, 2))  # 7.88
print(abs(-100))         # 100
print(math.sqrt(225))    # 15.0
```

13. Functions in Python

13.1 What is a Function?

A function is a named block of code that performs a specific task. It is defined once and can be called (used) multiple times in a program.

Importance and Advantages

- Reusability — Write once, use many times.
- Reduces Code Duplication — No need to repeat the same code.
- Easy Maintenance — Changes in one place apply everywhere.
- Better Readability — Programs become easier to understand.
- Modularity — Divide large programs into smaller, manageable parts.

13.2 Types of Functions

Type	Description and Examples
Built-in Functions	Pre-defined in Python: print(), input(), len(), sum(), abs()
User-defined Functions	Created by the programmer for specific needs

13.3 Defining a Function

A function is defined using the def keyword.

Syntax

```
def function_name(parameters):  
    "Optional docstring explaining the function"  
    # function body  
    return value # optional
```

Components:

- def — keyword to define a function
- function_name — the name of the function (follows variable naming rules)
- parameters — input values (optional)
- function body — code that runs when function is called
- return — sends back a result (optional)

Example — Simple function:

```
def greet():  
    print('Hello! Welcome to Python.')
```

13.4 Calling a Function

To run a function, you call it by writing its name followed by parentheses.

```
greet() # Calling the function  
# Output: Hello! Welcome to Python.
```

Example — Function with parameter:

```
def greet(name):  
    print(f'Hello, {name}!')
```

```
greet('Priya') # Hello, Priya!  
greet('Rohan') # Hello, Rohan!
```

14. Scope of Functions

14.1 What is Scope?

Scope refers to the area of a program where a variable can be accessed. Not all variables are available everywhere in a program.

14.2 Types of Scope

Local Scope

A variable created inside a function has local scope. It can only be accessed inside that function and does not exist outside it.

```
def my_func():  
    x = 10    # local variable  
    print(x)  # works fine
```

```
my_func()  
# print(x) ← This would cause NameError: x is not defined
```

Global Scope

A variable created outside all functions has global scope. It can be accessed anywhere in the program, both inside and outside functions.

```
y = 50    # global variable
```

```
def my_func():  
    print(y)  # can access global variable
```

```
my_func()    # 50  
print(y)     # 50
```

Using global keyword inside a function:

```
count = 0
```

```
def increment():  
    global count  
    count += 1
```

```
increment()  
increment()  
print(count)  # 2
```

14.3 Difference: Local vs Global Scope

Local Scope	Global Scope
Created inside a function	Created outside all functions
Only accessible within that function	Accessible throughout the program

Local Scope	Global Scope
Created when function is called	Created when program starts
Destroyed when function ends	Exists as long as program runs
Example: <code>def f(): x = 5</code>	Example: <code>x = 5</code> (at top level)

15. Function Parameters

15.1 What are Parameters?

Parameters are the variables listed in the function definition. They act as placeholders for the values (arguments) that will be passed when the function is called.

15.2 Types of Parameters

A. Required / Positional Parameters

These must be passed in the correct order and the exact number must match.

```
def add(a, b):  
    print(a + b)
```

```
add(10, 5)    # 15  
add(3, 7)    # 10
```

B. Default Parameters

A default value is assigned in the function definition. If not provided during the call, the default is used.

```
def greet(name, greeting='Hello'):  
    print(f'{greeting}, {name}!')
```

```
greet('Alice')          # Hello, Alice!  
greet('Bob', 'Good Morning') # Good Morning, Bob!
```

C. Keyword Parameters

Arguments are passed by name, so order does not matter.

```
def introduce(name, age):  
    print(f'Name: {name}, Age: {age}')
```

```
introduce(age=16, name='Riya') # Name: Riya, Age: 16
```

D. Variable Length Parameters (*args)

*args allows a function to accept any number of positional arguments as a tuple.

```
def total(*args):  
    print(sum(args))
```

```
total(1, 2, 3)    # 6  
total(10, 20, 30, 40) # 100
```

E. Variable Length Keyword Parameters (**kwargs)

**kwargs allows a function to accept any number of keyword arguments as a dictionary.

```
def show_info(**kwargs):  
    for key, val in kwargs.items():  
        print(f'{key}: {val}')
```



```
show_info(name='Sam', age=15, city='Kathmandu')  
# name: Sam  
# age: 15  
# city: Kathmandu
```

16. Function Arguments

16.1 What are Arguments?

Arguments are the actual values passed to the function when it is called. Parameters are in the definition; arguments are in the call.

16.2 Types of Arguments Passed

Argument Type	Example
Number	def f(x): → f(42)
String	def f(s): → f('hello')
Boolean	def f(b): → f(True)
List	def f(lst): → f([1,2,3])
Tuple	def f(t): → f((1,2,3))
Dictionary	def f(d): → f({'a':1})
Variable	x=5; def f(n): → f(x)
Expression	def f(n): → f(2+3*4)
Another Function	def f(fn): → f(len)

Comprehensive Example:

```
def display(data):  
    print('Received:', data)
```

```
display(100)          # Number  
display('Python')     # String  
display(True)         # Boolean  
display([1, 2, 3])    # List  
display({'name': 'Alice'}) # Dictionary
```

17. Function Return Type

17.1 return Statement

The return statement sends back a value from a function to the caller. After return, the function stops executing.

Syntax

```
def function_name():  
    return value
```

17.2 Returning Values

Returning a Single Value:

```
def square(n):  
    return n * n
```

```
result = square(5)  
print(result)  # 25
```

Returning Multiple Values:

```
def min_max(lst):  
    return min(lst), max(lst)
```

```
lo, hi = min_max([3, 1, 8, 5])  
print(lo, hi)  # 1 8
```

Returning a List:

```
def squares(n):  
    return [i**2 for i in range(1, n+1)]
```

```
print(squares(5))  # [1, 4, 9, 16, 25]
```

Returning a Dictionary:

```
def student_info(name, age):  
    return {'name': name, 'age': age}
```

```
info = student_info('Anil', 16)  
print(info)  # {'name': 'Anil', 'age': 16}
```

 **Exam Tip:** A function with no return statement returns None by default.

18. Passing Concepts in Functions: Mutable vs Immutable

When you pass a variable to a function, Python's behavior depends on whether the object is mutable or immutable.

18.1 Mutable Objects

Mutable objects CAN be changed after creation. When passed to a function, changes made inside the function AFFECT the original object.

- List, Dictionary, Set are mutable.

Example — List (mutable):

```
def add_item(my_list):  
    my_list.append(100)
```

```
nums = [1, 2, 3]  
add_item(nums)  
print(nums)  # [1, 2, 3, 100] ← original IS changed
```

18.2 Immutable Objects

Immutable objects CANNOT be changed after creation. When passed to a function, changes inside do NOT affect the original.

- Integer, Float, String, Tuple are immutable.

Example — Integer (immutable):

```
def change(x):  
    x = x + 10  
    print('Inside:', x)
```

```
n = 5  
change(n)  
print('Outside:', n)  # 5 ← original NOT changed
```

Mutable Objects	Immutable Objects
Can be changed	Cannot be changed
list, dict, set	int, float, str, tuple
Function changes affect original	Function changes do NOT affect original
Passed by reference (sort of)	Passed by value (sort of)

19. Dry Run and Program Tracing

19.1 What is a Dry Run?

A dry run is the process of manually tracing through a program step by step, tracking the value of each variable at each step, without running it on a computer. It helps understand what a program does and find errors.

Importance

- Helps understand program logic.
- Identifies bugs before running code.
- Common in exams — trace programs and fill dry run tables.

19.2 Sequential Flow — Dry Run

Program:

```
a = 5
b = 3
c = a + b
d = a * b
print(c, d)
```

Step	Statement	Variable Changed	Value
1	a = 5	a	5
2	b = 3	b	3
3	c = a + b	c	8
4	d = a * b	d	15
5	print(c, d)	—	Output: 8 15

19.3 Conditional Structure — Dry Run

Program:

```
x = 10
if x > 5:
    print('Big')
else:
    print('Small')
```

Step	Check	Action
1	x = 10	x is assigned 10
2	x > 5	10 > 5 → True
3	True branch	print('Big')

Step	Check	Action
Output	—	Big

19.4 Loop Structure — Dry Run

Program:

```
total = 0
for i in range(1, 5):
    total += i
print(total)
```

Iteration	i	total += i	total
Start	—	—	0
1st	1	0+1	1
2nd	2	1+2	3
3rd	3	3+3	6
4th	4	6+4	10
Output	—	print(10)	10

 **Exam Tip:** In exams, always create a dry run table with columns: Step/Iteration, Variable, Value, Output.

20. Modules, Packages and Libraries

20.1 Module

A module is a file containing Python code (functions, variables, classes) that can be imported and used in other programs. Module files have a .py extension.

Example: math, random, os, sys

20.2 from Keyword

The from keyword allows you to import specific functions from a module without importing the whole module.

Syntax

```
from module_name import function_name
```

Example:

```
from math import sqrt
print(sqrt(25)) # 5.0 (no need to write math.sqrt)
```

20.3 Package

A package is a directory (folder) containing multiple related modules. It must have a special file called `__init__.py` to be treated as a package.

Folder structure:

```
mypackage/
__init__.py
module1.py
module2.py
```

```
from mypackage import module1
```

20.4 Library

A library is a collection of modules and packages that provide ready-made tools for specific purposes.

Library	Purpose
math	Mathematical functions
random	Random number generation
pandas	Data analysis and manipulation
matplotlib	Data visualization and plotting
turtle	Graphics and drawing
os	Operating system interactions

Library	Purpose
datetime	Date and time operations

21. Importing Libraries

21.1 Import Entire Library

```
import math
print(math.sqrt(64)) # 8.0
print(math.pi)      # 3.14159...
```

21.2 Import Specific Function

```
from math import sqrt, pi
print(sqrt(81)) # 9.0
print(pi)      # 3.14159...
```

21.3 Import All Functions

```
from math import *
print(sqrt(100)) # 10.0 (no math. prefix needed)
```

Import Style	Usage
import math	math.sqrt(25) — module prefix required
from math import sqrt	sqrt(25) — no prefix needed
from math import *	sqrt(25), pi, etc. — all directly available

 **Exam Tip:** Avoid 'from module import *' in large programs — it can cause naming conflicts.

22. Popular Python Libraries

22.1 Math Library

The math library provides mathematical functions and constants.

Function / Constant	Description and Example
<code>math.sqrt(x)</code>	Square root: <code>math.sqrt(144) → 12.0</code>
<code>math.ceil(x)</code>	Round up: <code>math.ceil(4.3) → 5</code>
<code>math.floor(x)</code>	Round down: <code>math.floor(4.9) → 4</code>
<code>math.factorial(n)</code>	Factorial: <code>math.factorial(5) → 120</code>
<code>math.pi</code>	Value of π : <code>math.pi → 3.14159265...</code>

```
import math
print(math.sqrt(49))    # 7.0
print(math.ceil(3.1))   # 4
print(math.floor(3.9))  # 3
print(math.factorial(6)) # 720
print(math.pi)          # 3.141592653589793
```

22.2 Random Library

The random library generates random numbers and makes random selections.

Function	Description and Example
<code>random.random()</code>	Float between 0.0 and 1.0: 0.734...
<code>random.randint(a,b)</code>	Random integer between a and b (inclusive): <code>randint(1,6) → could be 3</code>
<code>random.randrange(start,stop,step)</code>	Random from range: <code>randrange(0,10,2) → 4</code>
<code>random.choice(seq)</code>	Random item from a list: <code>choice(['a','b','c']) → 'b'</code>
<code>random.shuffle(lst)</code>	Shuffle list in-place: <code>shuffle([1,2,3,4])</code>
<code>random.uniform(a,b)</code>	Random float between a and b: <code>uniform(1,5) → 2.7</code>

```
import random
print(random.randint(1, 100))    # e.g. 47
print(random.choice(['red','blue','green'])) # e.g. blue
```

```
lst = [1, 2, 3, 4, 5]
random.shuffle(lst)
print(lst) # e.g. [3, 1, 5, 2, 4]
```

22.3 Pandas Library

Pandas is a powerful data analysis library. It provides data structures like DataFrame and Series for manipulating structured data.

- Definition: An open-source library for data analysis and manipulation.
- Uses: Reading/writing CSV, Excel, JSON files; filtering, sorting, grouping data.
- Key feature: DataFrame — a 2D table similar to an Excel spreadsheet.

22.4 Turtle Library

The Turtle library provides a simple way to draw shapes and graphics by controlling a virtual turtle that moves on the screen.

- Definition: A graphics library based on a turtle that draws lines.
- Uses: Drawing shapes, patterns, creating simple animations.

22.5 Matplotlib Library

Matplotlib is the most popular Python library for creating charts and graphs (line charts, bar charts, pie charts, etc.).

- Definition: A plotting library for creating static, interactive, and animated visualizations.
- Uses: Data visualization, scientific plotting, creating publication-quality charts.

23. Graphics Using Turtle

Turtle Commands

Command	Description
forward(n) or fd(n)	Move forward n units
backward(n) or bk(n)	Move backward n units
left(angle) or lt(angle)	Turn left by angle degrees
right(angle) or rt(angle)	Turn right by angle degrees
penup() or pu()	Lift pen — move without drawing
pendown() or pd()	Put pen down — draw as turtle moves
color('name')	Set pen/fill color: color('red')
speed(n)	Set speed (0–10, 0 = fastest)
shape('name')	Set turtle shape: 'turtle', 'arrow', 'circle'
done()	Keep window open after drawing

Drawing Shapes

Line:

```
import turtle
turtle.forward(100)
turtle.done()
```

Square:

```
import turtle
for i in range(4):
    turtle.forward(100)
    turtle.right(90)
turtle.done()
```

Rectangle:

```
import turtle
for i in range(2):
    turtle.forward(200)
    turtle.right(90)
    turtle.forward(100)
    turtle.right(90)
turtle.done()
```

Triangle:

```
import turtle
for i in range(3):
```

```
turtle.forward(100)
turtle.left(120)
turtle.done()
```

Circle:

```
import turtle
turtle.circle(100)
turtle.done()
```

 **Exam Tip:** Always end turtle programs with `turtle.done()` to keep the drawing window open.

24. Error Handling

24.1 Errors and Exceptions

An error is a problem in the program that prevents it from running correctly. Python stops execution when it encounters an error. Exceptions are errors that occur during program execution (runtime).

24.2 Types of Errors

Error Type	Definition and Example
Syntax Error	Violation of Python grammar rules. Detected before running. Example: <code>print 'Hello'</code> (missing parentheses)
Runtime Error	Occurs during execution. Example: <code>10 / 0</code> → <code>ZeroDivisionError</code> , <code>int('abc')</code> → <code>ValueError</code>
Logical Error	Program runs but gives wrong output. Example: using <code>+</code> instead of <code>*</code> in a calculation

Syntax Error Example:

```
for i in range(5) # Missing colon → SyntaxError
```

Runtime Error Example:

```
x = int('hello') # ValueError: invalid literal
y = 10 / 0       # ZeroDivisionError
```

Logical Error Example:

```
area = length + width # Should be length * width
```

 **Exam Tip:** Syntax errors are caught before running. Runtime and logical errors occur during execution.

25. Try-Except: Exception Handling

25.1 try-except Block

The try-except block is used to handle runtime errors gracefully, preventing the program from crashing.

```
try:
    # code that might cause an error
except ExceptionType:
    # code to handle the error
```

Example:

```
try:
    num = int(input('Enter a number: '))
    print(100 / num)
except ZeroDivisionError:
    print('Error: Cannot divide by zero!')
except ValueError:
    print('Error: Please enter a valid number!')
```

25.2 Multiple Except Blocks

You can have multiple except blocks to handle different types of exceptions differently.

```
try:
    a = int(input('Enter a: '))
    b = int(input('Enter b: '))
    print(a / b)
except ValueError:
    print('Invalid input! Enter integers only.')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print('Unexpected error:', e)
```

25.3 else with try-except

The else block runs only if NO exception occurred in the try block.

```
try:
    n = int(input('Enter a number: '))
    result = 100 / n
except ZeroDivisionError:
    print('Cannot divide by zero!')
else:
    print('Result:', result) # runs only if no error
```

25.4 finally Block

The finally block ALWAYS runs, whether an exception occurred or not. Used for cleanup actions like closing files.

```
try:
    f = open('data.txt', 'r')
```

```
content = f.read()
except FileNotFoundError:
    print('File not found!')
finally:
    print('This always runs — cleanup done.')
```

Block	When it Runs
try	Always — this is the code to monitor
except	Only when the specified exception occurs
else	Only when NO exception occurs
finally	ALWAYS — with or without exceptions

25.5 Blank Input Error

When user enters nothing (empty string) and the program tries to convert it, a `ValueError` occurs.

```
try:
    age = int(input('Enter age: '))
    print('Your age is', age)
except ValueError:
    print('Error: Please enter a valid number. Do not leave blank!')
```


26. File Handling

26.1 What is File Handling?

File handling is the ability of a program to create, open, read, write, and close files stored on the computer's disk. Without file handling, data is lost when the program ends.

Importance

- Store data permanently (not lost when program stops).
- Share data between programs.
- Process large amounts of data stored in files.

26.2 File Modes

Mode	Meaning
'r' (Read)	Opens file for reading only. Error if file doesn't exist.
'w' (Write)	Opens file for writing. Creates new file or OVERWRITES existing content.
'a' (Append)	Opens file for appending. Adds new content at the END without deleting existing.
'r+' (Read and Write)	Opens for both reading and writing. File must exist.
'w+' (Write and Read)	Opens for reading and writing. Creates or overwrites.
'a+' (Append and Read)	Opens for reading and appending.

 **Exam Tip:** 'w' mode deletes all existing content. Use 'a' mode if you want to keep the old data.

27. Basic File Operations

Opening a File — open()

```
file_object = open('filename.txt', 'mode')
```

Examples:

```
f = open('data.txt', 'r') # open for reading
f = open('data.txt', 'w') # open for writing
f = open('data.txt', 'a') # open for appending
```

Creating and Writing a File — write()

```
f = open('student.txt', 'w')
f.write('Name: Alice\n')
f.write('Age: 16\n')
f.write('Grade: A\n')
f.close()
```

Reading a File — read()

```
f = open('student.txt', 'r')
content = f.read()
print(content)
f.close()
```

Closing a File — close()

Always close a file after operations to free up system resources.

```
f.close()
```

Best Practice — with statement

Using with automatically closes the file when the block ends, even if an error occurs.

```
with open('data.txt', 'r') as f:
    content = f.read()
    print(content)
# File is automatically closed after this block
```

 **Exam Tip:** Always use the with statement for file handling — it is safer and cleaner.

28. Reading Data from File

readline()

Reads one line at a time from the file.

```
f = open('student.txt', 'r')
line1 = f.readline()
print(line1)    # Name: Alice
line2 = f.readline()
print(line2)    # Age: 16
f.close()
```

readlines()

Reads ALL lines and returns them as a list where each item is one line.

```
f = open('student.txt', 'r')
lines = f.readlines()
for line in lines:
    print(line, end="")
f.close()
```

Method	What it Returns
read()	Entire file as one single string
readline()	One line as a string (each call gets next line)
readlines()	All lines as a list of strings

29. CSV File Handling

29.1 What is a CSV File?

CSV stands for Comma-Separated Values. It is a plain text file where data is stored in rows, with each value separated by a comma. CSV files are widely used to store tabular data like Excel spreadsheets.

Example CSV content:

```
Name,Age,Grade
Alice,16,A
Bob,15,B
Carol,17,A
```

Advantages of CSV

- Simple, lightweight format.
- Can be opened in Excel and other spreadsheet tools.
- Universal format — works with almost all data tools.

29.2 Writing a CSV File

```
import csv
```

```
with open('students.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Name', 'Age', 'Grade']) # header
    writer.writerow(['Alice', 16, 'A'])
    writer.writerow(['Bob', 15, 'B'])
    writer.writerow(['Carol', 17, 'A'])
```

 **Exam Tip:** Always use `newline=""` when opening CSV files to prevent extra blank lines on Windows.

29.3 Reading a CSV File

```
import csv
```

```
with open('students.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

```
# Output:
# ['Name', 'Age', 'Grade']
# ['Alice', '16', 'A']
# ['Bob', '15', 'B']
# ['Carol', '17', 'A']
```

30. Pandas File Handling

30.1 Introduction

Pandas is a powerful Python library for data analysis. It provides the DataFrame structure — a 2D table like an Excel sheet — to work with structured data easily.

Advantages

- Easy to read/write CSV, Excel, JSON files.
- Powerful data analysis functions.
- Handles missing data automatically.

```
import pandas as pd
```

30.2 Pandas File Functions

Function	Purpose
pd.read_csv('file.csv')	Read a CSV file into a DataFrame
pd.read_excel('file.xlsx')	Read an Excel file into a DataFrame
pd.read_json('file.json')	Read a JSON file into a DataFrame
pd.read_html('url')	Read HTML tables from a webpage
pd.DataFrame(data)	Create a DataFrame from a dictionary or list
df.to_csv('file.csv')	Write DataFrame to a CSV file
df.to_excel('file.xlsx')	Write DataFrame to an Excel file

31. CSV Handling Using Pandas

Creating CSV File with Pandas

```
import pandas as pd
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Carol'],  
    'Age': [16, 15, 17],  
    'Grade': ['A', 'B', 'A']  
}
```

```
df = pd.DataFrame(data)  
df.to_csv('students.csv', index=False)  
print('CSV file created successfully!')
```

Reading CSV with Pandas

```
import pandas as pd
```

```
df = pd.read_csv('students.csv')  
print(df)  
# Output shows the data in table format
```

Data Analysis Functions

Function / Attribute	Description and Example
head(n)	Show first n rows (default 5): df.head()
tail(n)	Show last n rows (default 5): df.tail(3)
shape	Tuple of (rows, columns): df.shape → (3, 3)
columns	List column names: df.columns → ['Name', 'Age', 'Grade']
mean()	Mean of numeric columns: df['Age'].mean() → 16.0
describe()	Statistical summary (count, mean, std, min, max)

Full Example with Analysis:

```
import pandas as pd
```

```
df = pd.read_csv('students.csv')  
print('First 3 rows:')  
print(df.head(3))  
print('\nShape:', df.shape)  
print('Columns:', df.columns.tolist())  
print('Average Age:', df['Age'].mean())
```

```
print("\nFull Summary:")  
print(df.describe())
```

32. Data Visualization

32.1 What is Data Visualization?

Data visualization is the graphical representation of data and information using charts, graphs, maps, and other visual tools. It makes complex data easy to understand and analyze.

Advantages

- Makes patterns and trends easy to see.
- Simplifies large and complex datasets.
- Helps in making better decisions.
- Easy to communicate findings to others.
- Visual data is processed faster by the human brain.

33. Matplotlib

33.1 Introduction

Matplotlib is the most widely used Python library for creating visualizations. Its main module is pyplot, which provides MATLAB-like functions for plotting.

```
import matplotlib.pyplot as plt
```

33.2 Common Methods

Method	Purpose and Example
<code>plt.plot(x, y)</code>	Draw a line chart
<code>plt.title('text')</code>	Set chart title
<code>plt.xlabel('text')</code>	Label the x-axis
<code>plt.ylabel('text')</code>	Label the y-axis
<code>plt.show()</code>	Display the chart
<code>plt.grid(True)</code>	Add grid lines
<code>plt.legend()</code>	Show legend for labelled plots
<code>plt.xlim(min, max)</code>	Set x-axis range
<code>plt.ylim(min, max)</code>	Set y-axis range

34. Line Chart

Definition

A line chart displays data as a series of points connected by straight lines. It is used to show trends over time or changes in data.

Applications

- Stock price over time.
- Temperature changes during a week.
- Student performance across exams.

Example — Line Chart

```
import matplotlib.pyplot as plt
```

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']  
sales = [1200, 1500, 1300, 1700, 1600]
```

```
plt.plot(months, sales, color='blue', marker='o', linewidth=2)  
plt.title('Monthly Sales')  
plt.xlabel('Month')  
plt.ylabel('Sales (Rs)')  
plt.grid(True)  
plt.show()
```

 **Exam Tip:** Use marker='o' to show data points clearly on the line chart.

35. Pie Chart

Definition

A pie chart is a circular chart divided into slices, where each slice represents the proportion of a category relative to the total. It is used to show percentages and parts of a whole.

Methods Used

Parameter / Method	Description
plt.pie(values)	Draw the pie chart
labels	Category names for each slice
autopct='%1.1f%%'	Show percentage on each slice
explode	Pull out a slice: explode=(0.1,0,0,0)
shadow=True	Add shadow effect
startangle=90	Starting angle for first slice
colors	List of colors for slices

Example — Pie Chart

```
import matplotlib.pyplot as plt

subjects = ['Math', 'Science', 'English', 'Nepali']
marks    = [85, 90, 78, 88]
explode  = (0.1, 0, 0, 0) # pull out Math slice

plt.pie(marks, labels=subjects, autopct='%1.1f%%',
        explode=explode, shadow=True, startangle=90)
plt.title('Marks Distribution')
plt.show()
```

36. Bar Chart

Definition

A bar chart displays data as rectangular bars. Each bar's height (or length) represents the value of the data. Used to compare categories.

Example — Basic Bar Chart

```
import matplotlib.pyplot as plt

subjects = ['Math', 'Science', 'English', 'Nepali']
marks    = [85, 92, 78, 88]

plt.bar(subjects, marks, color='steelblue', width=0.5, label='Marks')
plt.title('Subject-wise Marks')
plt.xlabel('Subjects')
plt.ylabel('Marks')
plt.legend()
plt.show()
```

Bar Chart Customization

Multiple Colors:

```
colors = ['red', 'green', 'blue', 'orange']
plt.bar(subjects, marks, color=colors)
```

Adding Labels on Bars:

```
bars = plt.bar(subjects, marks, color='skyblue')
for bar in bars:
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 1,
             str(int(bar.get_height())),
             ha='center', va='bottom')
```

Chart Type	Best Used For
Line Chart	Showing trends over time
Pie Chart	Showing parts of a whole / percentages
Bar Chart	Comparing values across categories

37. Animation

37.1 What is Animation?

Animation is the process of creating the illusion of movement by displaying a sequence of images or frames rapidly in succession. In computing, animation brings life to programs and data visualizations.

Importance

- Makes applications more engaging and interactive.
- Helps visualize data changes over time.
- Used in games, simulations, educational tools.

37.2 Types of Animation

2D Animation

2D animation works in a two-dimensional flat space (X and Y axes only). Objects move, rotate, or change in a flat plane.

- Definition: Animation in two dimensions — height and width.
- Features: Simple to create, less resource-intensive, cartoons and flat graphics.
- Examples: Animated GIFs, Turtle graphics animations, matplotlib animations.

3D Animation

3D animation adds depth (Z axis) to create realistic, three-dimensional objects that can be viewed from any angle.

- Definition: Animation in three dimensions — height, width, and depth.
- Features: Realistic appearance, complex rendering, used in games and films.
- Examples: Blender animations, 3D game characters, CGI films.

2D Animation	3D Animation
Flat, two-dimensional	Three-dimensional with depth
Simpler to create	More complex and resource-heavy
Cartoons, GIFs	Movies, video games, simulations
Uses X and Y axes	Uses X, Y, and Z axes

QUICK REVISION SUMMARY — CHAPTER 4 PYTHON

Topic	Key Point to Remember
Python	High-level, interpreted, platform-independent, open-source
input()	Always returns string — use int()/float() to convert
Data Types	int, float, str, bool, list, tuple, set, dict, None
Arithmetic Ops	// = floor division, % = modulus, ** = exponent
f-Strings	f'Hello {name}' — expressions allowed inside { }
if-elif-else	Check multiple conditions in order
List	Mutable, ordered, allows duplicates, uses []
Tuple	Immutable, ordered, allows duplicates, uses ()
Dictionary	Key-value pairs, keys unique, uses { }
range()	range(stop), range(start,stop), range(start,stop,step)
for loop	Iterates over sequence — used when iterations are known
while loop	Runs until condition False — used when iterations unknown
break	Exits loop immediately
continue	Skips current iteration, continues loop
else with loop	Runs when loop completes normally (no break)
*args	Variable positional arguments → tuple
**kwargs	Variable keyword arguments → dictionary
return	Sends value back from function; can return multiple values
Local scope	Variable inside function — only accessible there
Global scope	Variable outside function — accessible everywhere
Mutable	list/dict/set — changes inside function affect original
Immutable	int/float/str/tuple — changes inside function don't affect original

Topic	Key Point to Remember
try-except	Handle runtime errors gracefully
finally	Always runs — for cleanup
File 'w' mode	Creates/overwrites file — DANGER: deletes existing data
File 'a' mode	Appends to end — preserves existing data
CSV	Comma-Separated Values — import csv to handle
Pandas	DataFrame, read_csv(), to_csv(), head(), describe()
Line Chart	plt.plot() — trends over time
Bar Chart	plt.bar() — compare categories
Pie Chart	plt.pie() — parts of a whole / percentages